ICS Tutorial 2

Venkata Sreekar

March 2026

Part 1

Question 1

(a)
- **Parser Module**
  The **Parser** is responsible for reading and breaking down the input code instruction by instruction.
  - `hasMoreCommands()`: Checks if there are still commands left to process in the input and returns a boolean accordingly.
  - `advance()`: Moves on to the next command and sets it as the current command to work with.
  - `commandType()`: Tells us what kind of command we're dealing with (*A_COMMAND*, *C_COMMAND*, or *L_COMMAND*).
  - `symbol()`: Returns the symbol or decimal string associated with an *A* or *L* command.
  - `dest(), comp(), jump()`: Each of these returns the symbolic mnemonic for the corresponding field of a *C*-instruction.
- **Code Module**
  The **Code** module handles the translation of Hack assembly mnemonics into their binary bit-string equivalents.
  - `dest(mnemonic)`: Returns the 3-bit binary code for the destination field.
  - `comp(mnemonic)`: Returns the 7-bit binary code (including the *a*-bit) for the computation field.
  - `jump(mnemonic)`: Returns the 3-bit binary code for the jump field.
- **SymbolTable Module**
  The **SymbolTable** keeps track of the mapping between symbolic labels or variables and their corresponding numeric addresses.
  - `addEntry(symbol, address)`: Adds a new symbol-address pair into the table.
  - `contains(symbol)`: Returns a boolean telling us whether a given symbol already exists in the table.
  - `getAddress(symbol)`: Looks up and returns the memory address linked to a given symbol.

(b) The RAM addresses from 0-15 are occupied by the predefined symbols (R0 - R15).
Hence, If we want to make our new custom variable, we will allocate them from address 16

Part 2

Question 2

(a) To store each 3×3 matrix we need 9 registers (1 for each element).
As there are two 3×3 matrices (A and B) and output matrix (C) to be stored,

$$9 \times 3 = 27$$

we need a total of **27 registers** to store matrices A, B, C in matrix multiplication $C = A \times B$

(b) As we don't have an instruction for multiplication in ALU, we cannot directly implement multiplication.

Instead, we treat multiplication as repeated addition and add the `multiplier` to itself `multiplicand` number of times to implement the multiplication.

We implement this with a loop `MUL LOOP` in which, we add the `multiplier` to itself and end the `MUL LOOP` when we have added `multiplicand` number of times.

To reduce the runtime, choose the smaller number as multiplicand and bigger one as multiplier. (the iterations for multiplication will be smaller).

(c)
- **Static Initialization**
  We have to hardcode the every elements of the 2 matrices before we implement the matrix multiplication.
  And, we initialize the values of the matrix C inside the loop (this is common in both static and dynamic initialization).
- **Dynamic Initialization**
  In dynamic initialisation, the values of matrices A and B are not hardcoded into the program but are instead supplied at runtime.
  In the Hack environment, this is achieved by using a test script (`.tst` file) that uses `set RAM[100]`, `set RAM[101]`, ... commands to load matrix values into the designated memory locations before execution begins. This means the same assembled binary (`MatMul.hack`) can operate on any input matrices without needing to be reassembled. The program itself contains no initialization instructions for A and B — it simply assumes those RAM addresses are already populated when the computation loop starts.
  Alternatively, a more advanced dynamic approach would involve polling the keyboard memory map at `RAM[24576]` and reading values digit by digit at runtime, though this requires additional assembly infrastructure such as a decimal parser and key-release detection loop. The primary advantage of dynamic initialisation over static is flexibility — matrix values can change with every run, making the program reusable and testable with multiple inputs.

Part 3

Question 3

(a) while multiplying matrices A, B, we multiply every row of A with every column of B.
So if we store rows of A together and columns of B together, we can multiply easier (simply doing A = A + 1 for moving to next element multiplication).

If we store rows together in both of the matrices, while finding element wise multiplication of rows of A with columns of B, we have to move pointer by l (number of columns in the matrix B) each time

So, yes, choice of storing matrix A row-major and matrix B column-major affects the efficiency of assembly program (In fact this is better than other ways of storing).